



Agentes de IA e suas ferramentas

SAEP 2026

Prof. Dr. Armando L. Keller



Agenda



Agenda

- Apresentação (Teoria)
- Demonstração de exemplos
- Tempo para prática e exercícios (hands on)



Apresentação

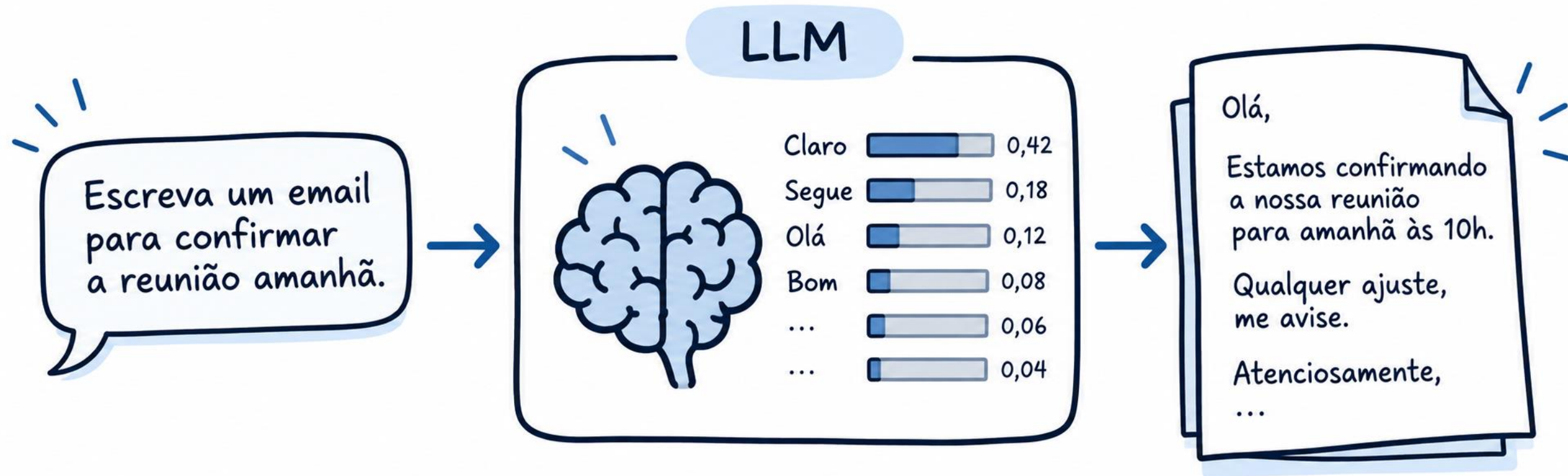
O que são agentes de IA?



LLMs (Large Language Models)



Os LLMs são geradores de texto baseados em probabilidades.





Usando LLMs

O modo mais comum de utilizar um LLM é através das interfaces de empresas como OpenAI (ChatGPT), Anthropic (Claude), Google (Gemini), entre outras.

Neste caso, normalmente tem-se modelos com pesos proprietários sendo executados em servidores com grande capacidade de processamento.

A interface do chat possui o seu próprio harness, de forma que o mesmo modelo tenha respostas diferentes em diferentes interfaces.



Usando LLMs

O acesso a estes modelos executados nos servidores externos também pode ser realizado através de uma API que normalmente possui uma cobrança por tokens de entrada e tokens de saída. É necessária a compra de créditos ou cadastro de meio de pagamento.

A vantagem é que pode-se utilizar um computador com configurações simples, e o processamento pesado fica nos servidores. Mas isto vem com um custo e com a necessidade de estar conectado à Internet.



Usando LLMs

Também é possível executar os modelos localmente, mas o tamanho do modelo é limitado pela quantidade de VRAM da GPU.

Para isto, existem diversas formas de executar, entre as principais destacam-se o LM Studio (Bionic) e o Ollama.

Nesta oficina, vamos utilizar o LM Studio (<https://lmstudio.ai/>) pois além de ter mais controle sobre o runtime, possui integração com o catálogo do HuggingFace.



Usando LLMs

Apesar das APIs poderem ser utilizadas com qualquer linguagem de programação, os principais frameworks como LangChain possuem suporte a Python e Typescript.

Nesta oficina vamos utilizar a linguagem Python e o framework LangChain.

Por motivos de organização, vamos utilizar o uv (<https://docs.astral.sh/uv/>) para gerenciar o ambiente virtual.



Configurando o ambiente



LM Studio (Bionic)

Como vamos executar o modelo localmente, precisamos abrir o LM Studio, e verificar se o modelo está sendo carregado corretamente.

Podemos utilizar qualquer modelo com suporte a ferramentas que caiba na GPU (menores que 4 GB no caso dos computadores do laboratório).

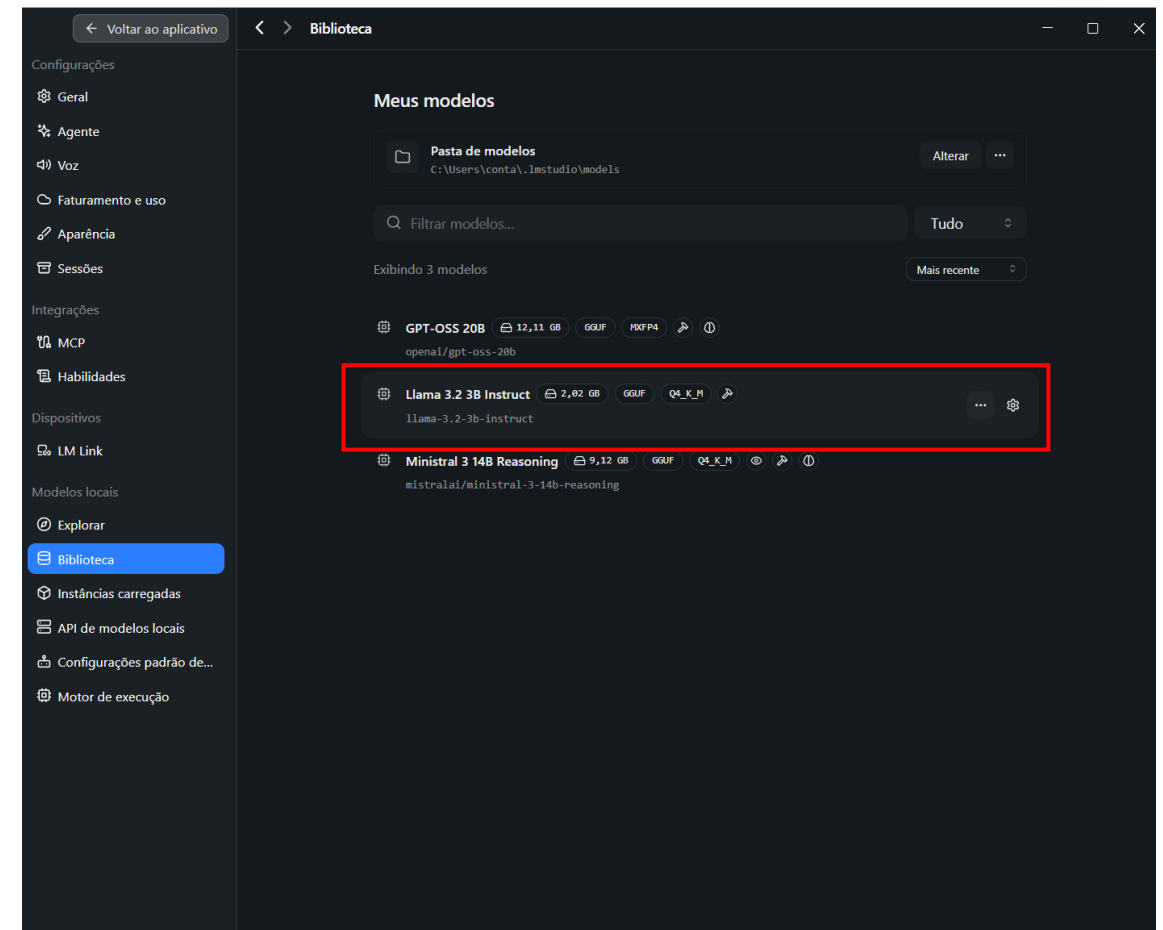
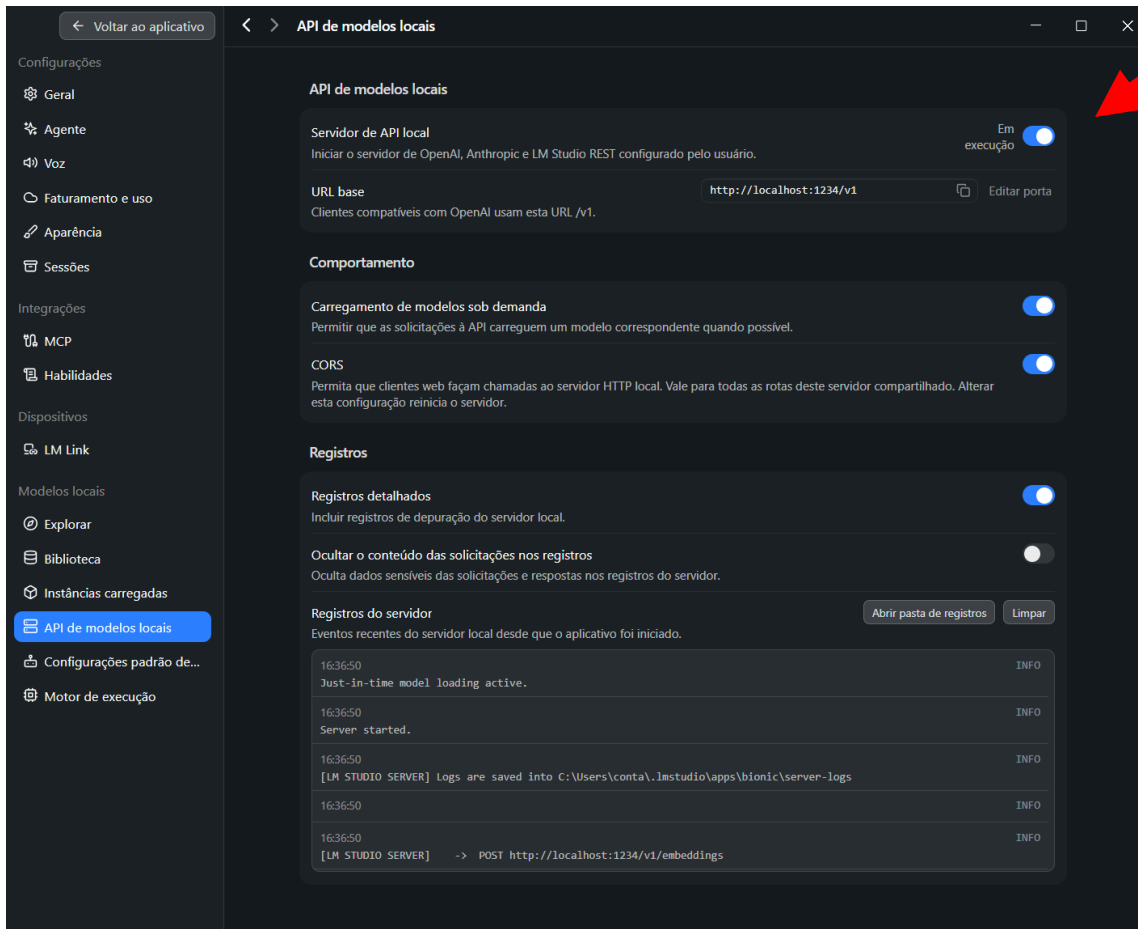
Recomendações:

- Llama-3.2-3B-Instruct
- Qwen2.5-3B-Instruct



LM Studio (Bionic)

Em configurações, verifique se a API está habilitada



Códigos dos exemplos



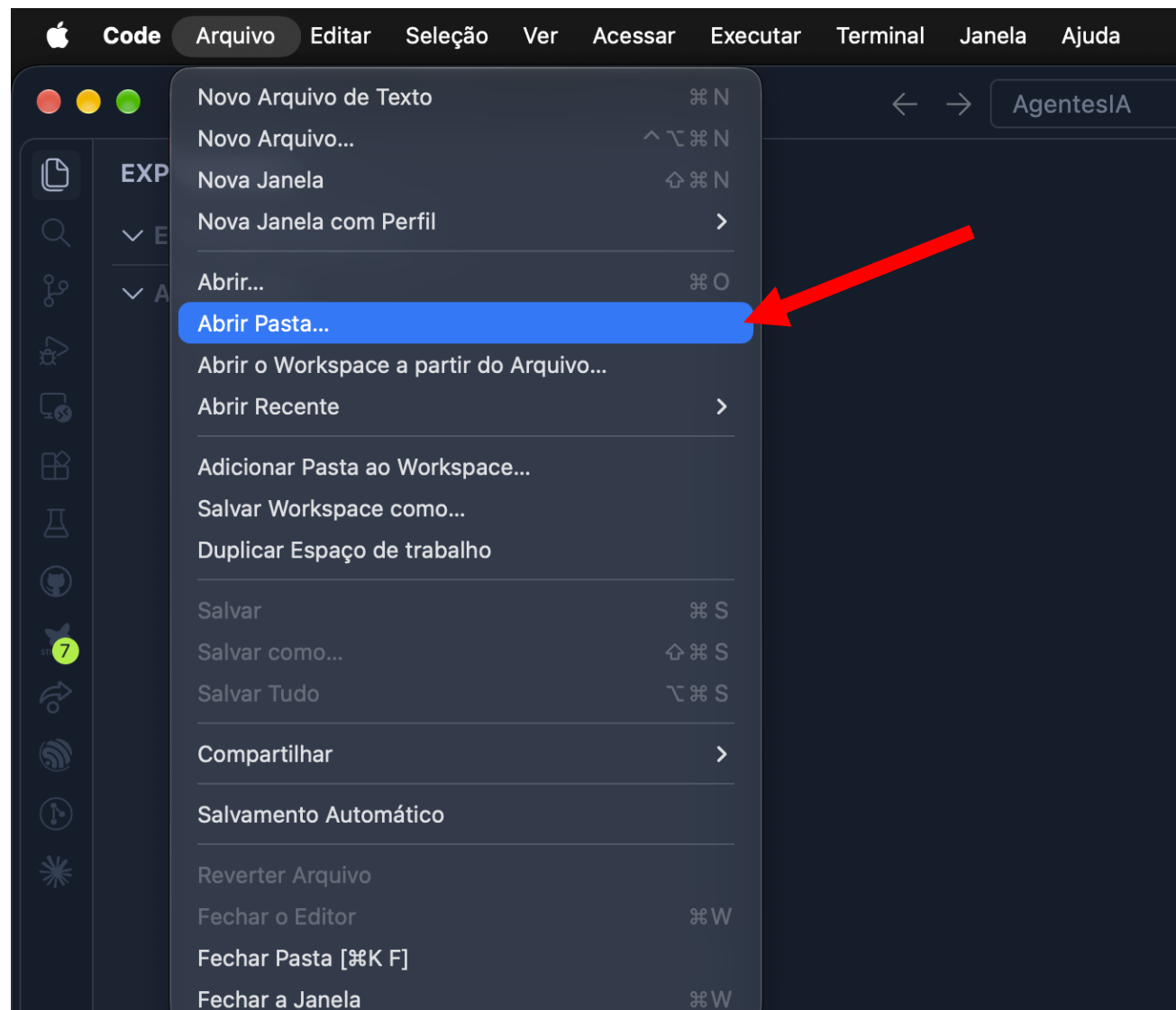
Os códigos dos exemplos estão disponíveis em
<https://github.com/armandokeller/SAEP2026-Agentes-IA-e-Ferramentas>



Visual Studio Code



Abra o Visual Studio Code, e abra a pasta com os códigos de exemplo.





Primeiro contato

Exemplo 0 - Testando o ambiente



O primeiro exemplo apenas valida a configuração do ambiente.

Para executar, abra o terminal no VS Code e execute o comando:

```
uv run ex0-testar-ambiente.py
```

Exemplo 1 - Chat mínimo

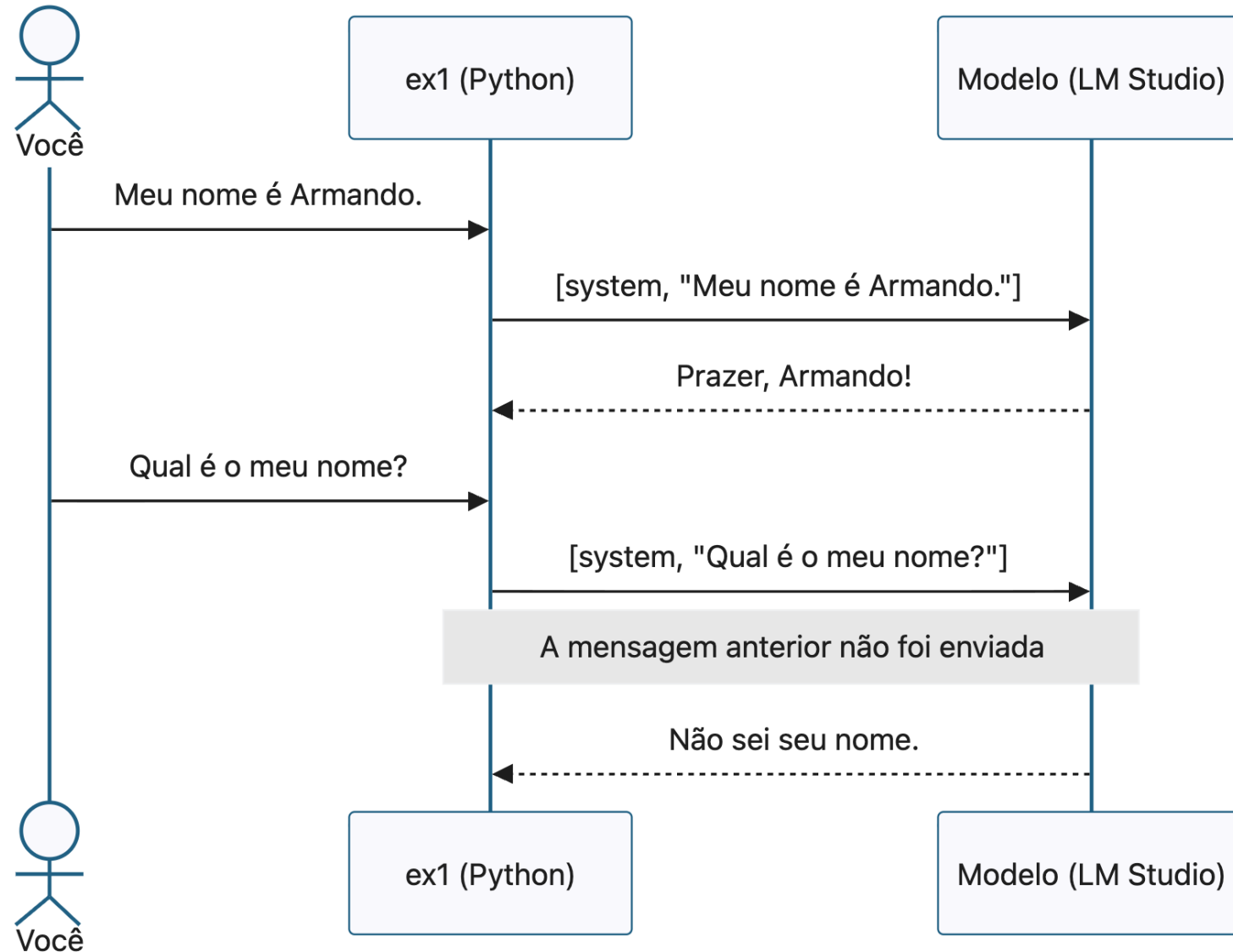


Neste exemplo, temos apenas a ligação da entrada do usuário com o modelo LLM e sua resposta, sem histórico de mensagens e sem guardar contexto.

Toda a informação está restrita à data de corte do treinamento do modelo.



Exemplo 1 - Chat mínimo





Adicionando memória

Exemplo 2 - Chat histórico



Para que o agente lembre do contexto, precisamos gerenciar a memória. Isto pode ser feito enviando as mensagens anteriores também.

Existem diferentes papéis de mensagens:

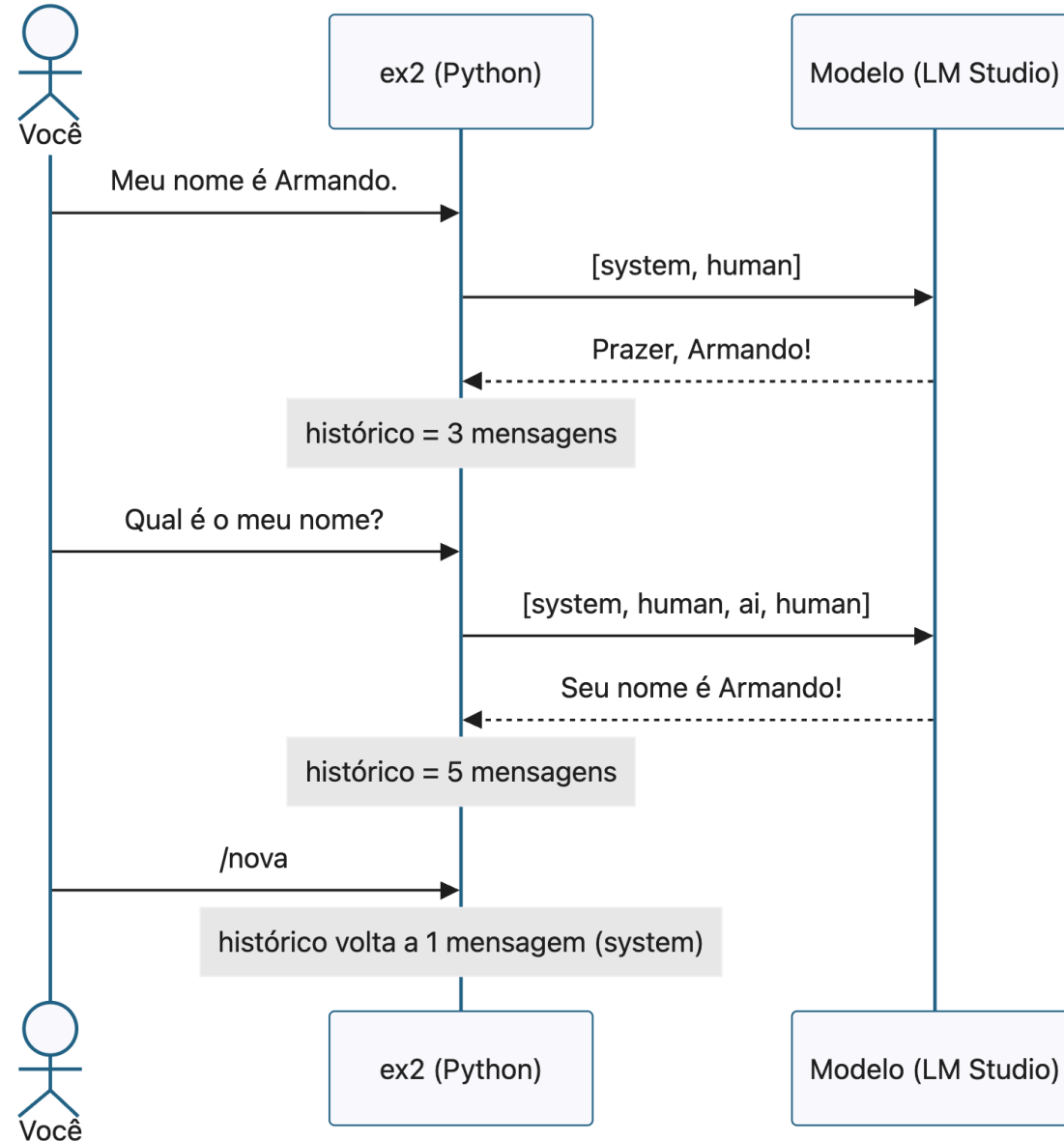
 System Message

 Human Message

 AI Message

 Tool Message

Exemplo 2 - Chat histórico





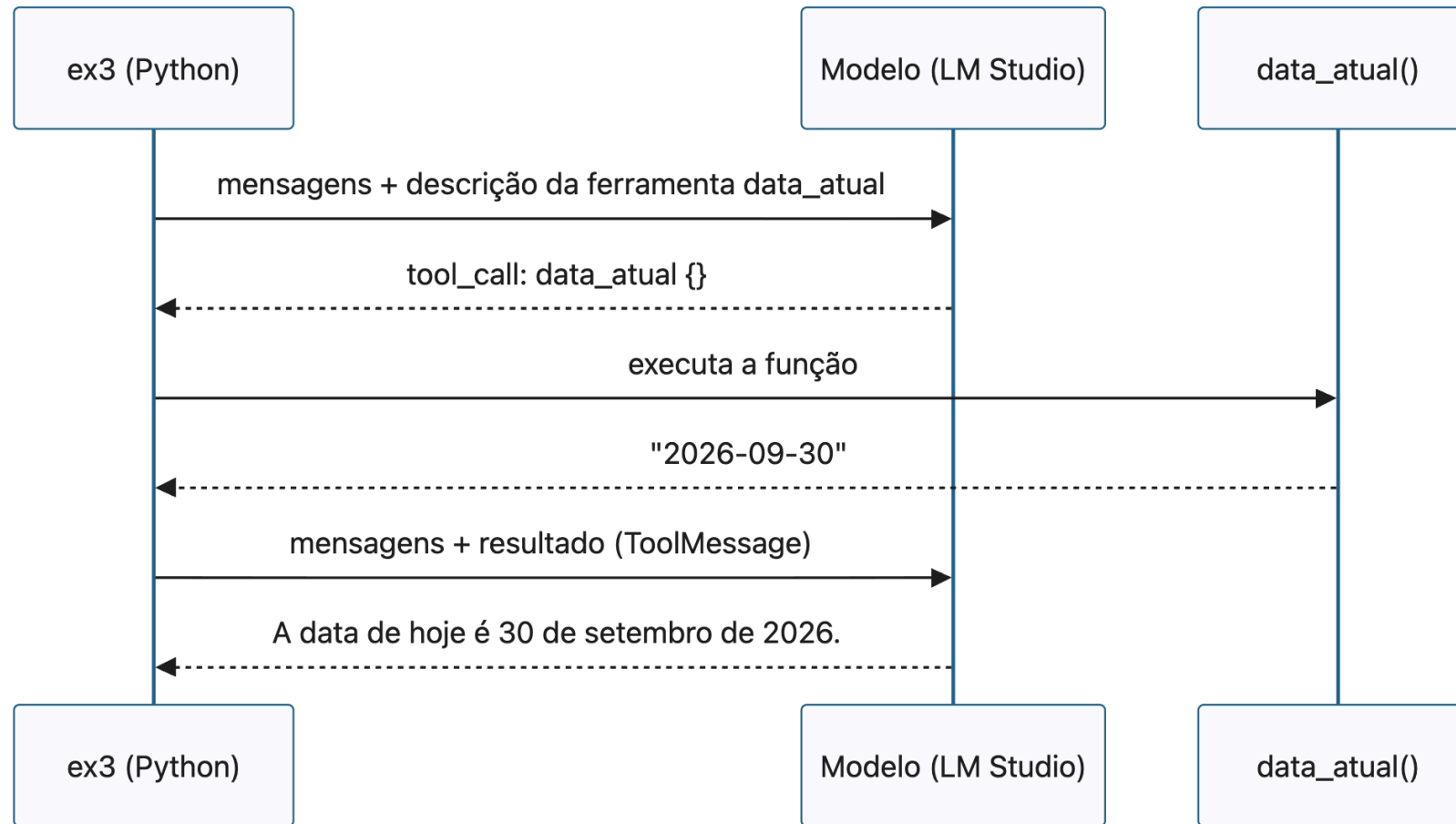
Adicionando herramientas

Exemplo 3 - Chat com ferramentas

Os LLMs são limitados aos dados de seu treinamento. Para adicionarmos a conexão com o mundo real, podemos permitir o uso de ferramentas.

Ferramentas nada mais são do que funções que o agente pode chamar, e tratar os seus resultados.

Exemplo 3 - Chat com ferramentas





Chat loop: um agente mínimo

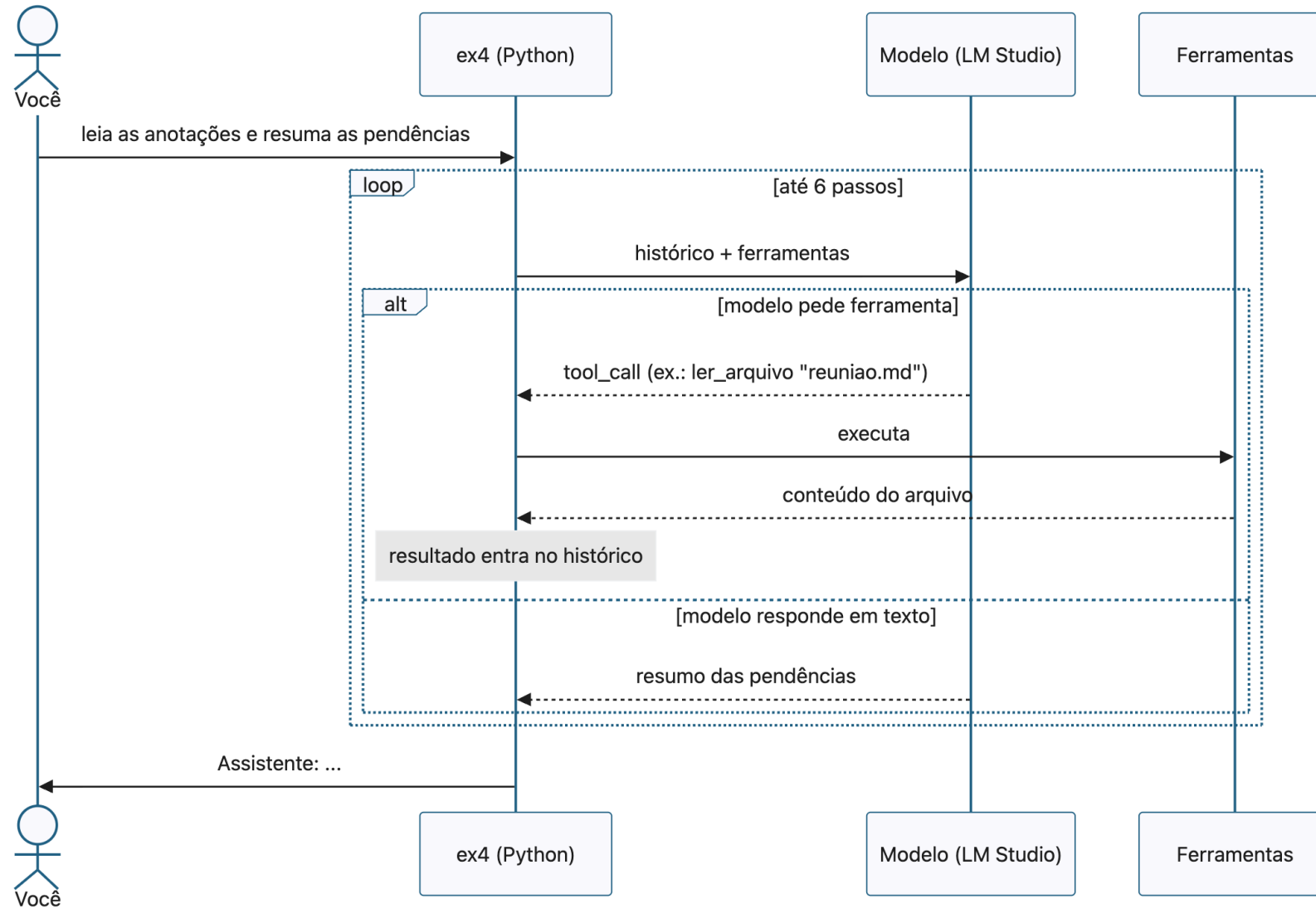
Exemplo 4 - Chat loop (um agente)



Neste exemplo vamos ter um agente mínimo, que recebe uma instrução do usuário e consegue executar uma cadeia de ações incluindo chamadas de ferramentas e interação entre elas.

Por exemplo, ao solicitar para resumir informações de arquivos, ele pode listar os arquivos, ler os arquivos, extrair o texto, resumir para só então devolver a resposta para o usuário.

Exemplo 4 - Chat loop (um agente)





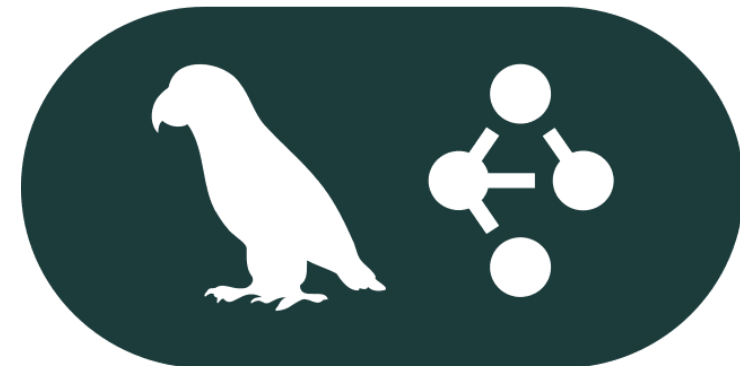
Grafos

Exemplo 5 - Grafos

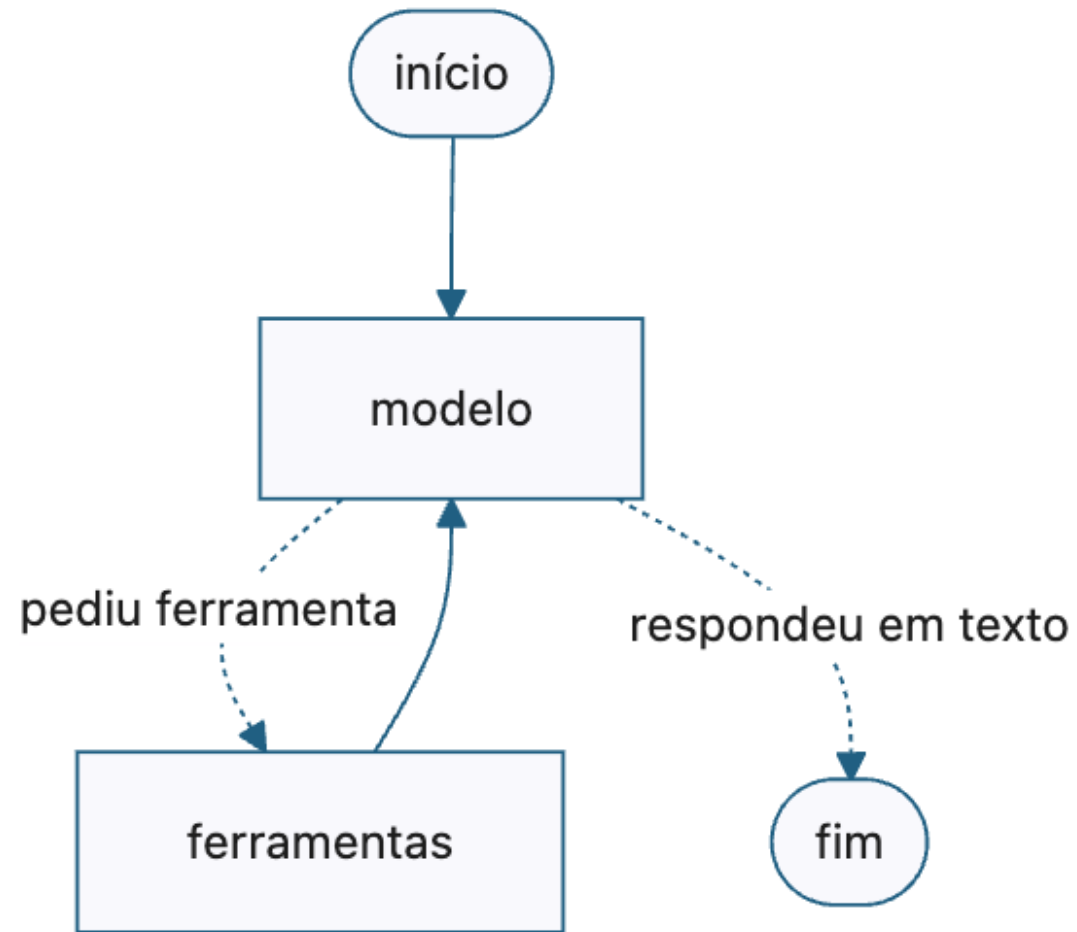


Deixar o agente decidir o fluxo, além de ter resultados pouco previsíveis, pode consumir uma quantidade grande de tokens.

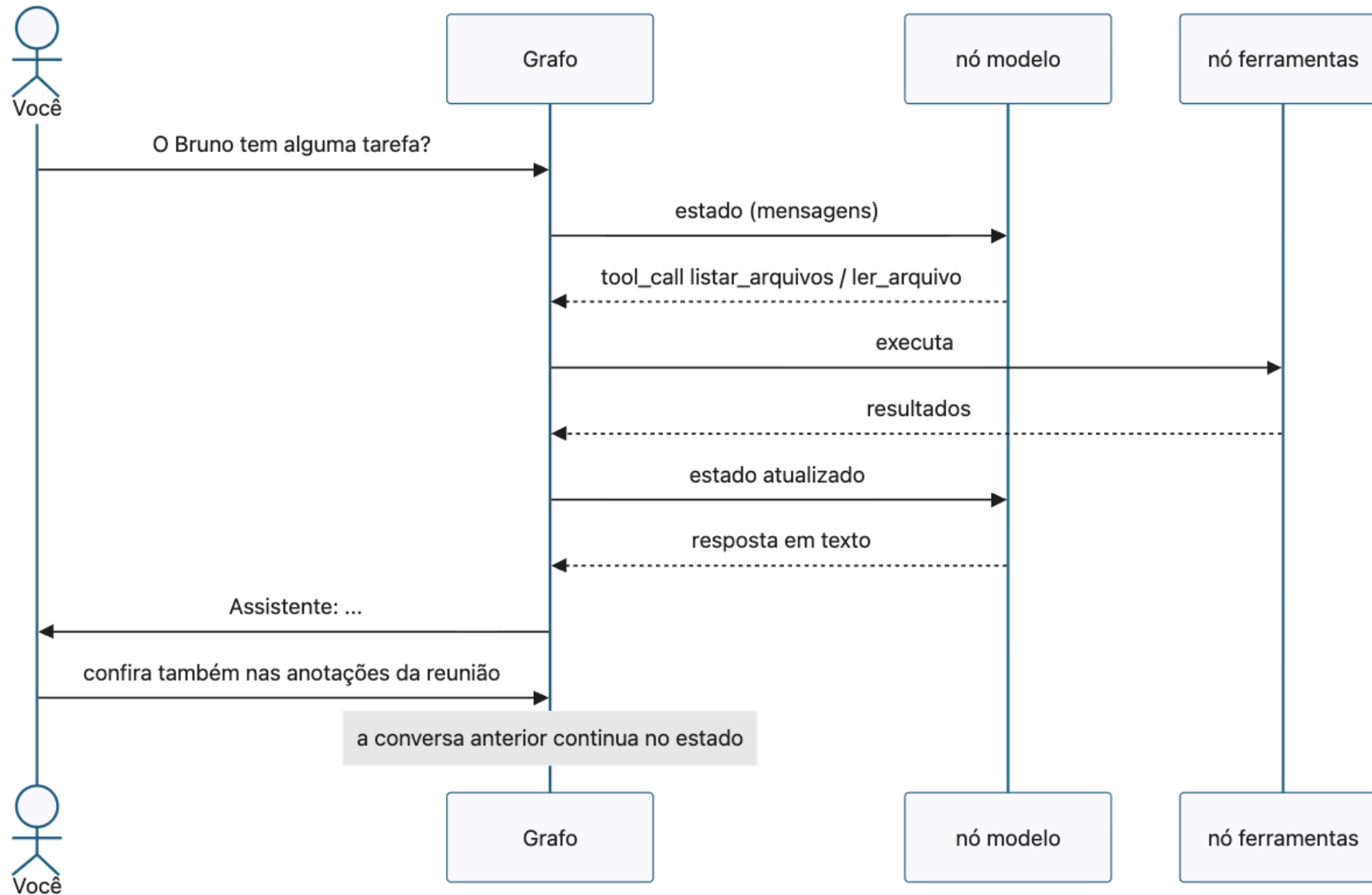
Para um controle mais preciso do comportamento do agente, ele pode ser construído em forma de grafo utilizando o framework LangGraph.



Exemplo 5 - Grafos



Exemplo 5 - Grafos





Ferramentas MCP

Exemplo 6 - Ferramentas MCP

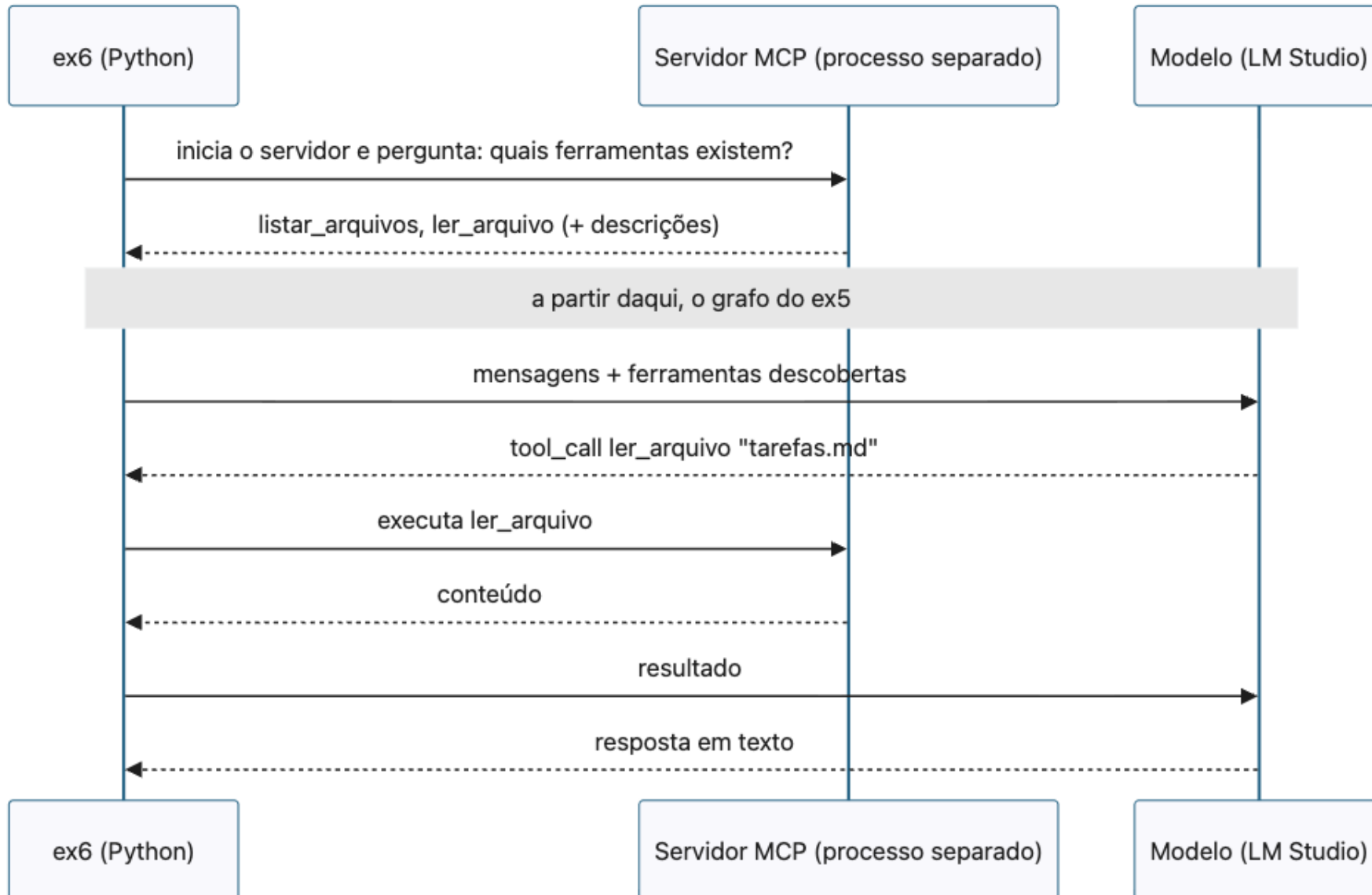


A Anthropic desenvolveu um formato padrão para disponibilização de ferramentas, o MCP (Model Context Protocol), onde as ferramentas são disponibilizadas por um servidor (local ou remoto), seguindo um contrato definido entre o agente e o servidor.

Com isto, é possível conectar a ferramentas de terceiros, assim como conectar as suas próprias ferramentas em outros agentes.

Para a criação das ferramentas vamos utilizar o FastMCP.

Exemplo 6 - Ferramentas MCP





HIL (Human in the loop)

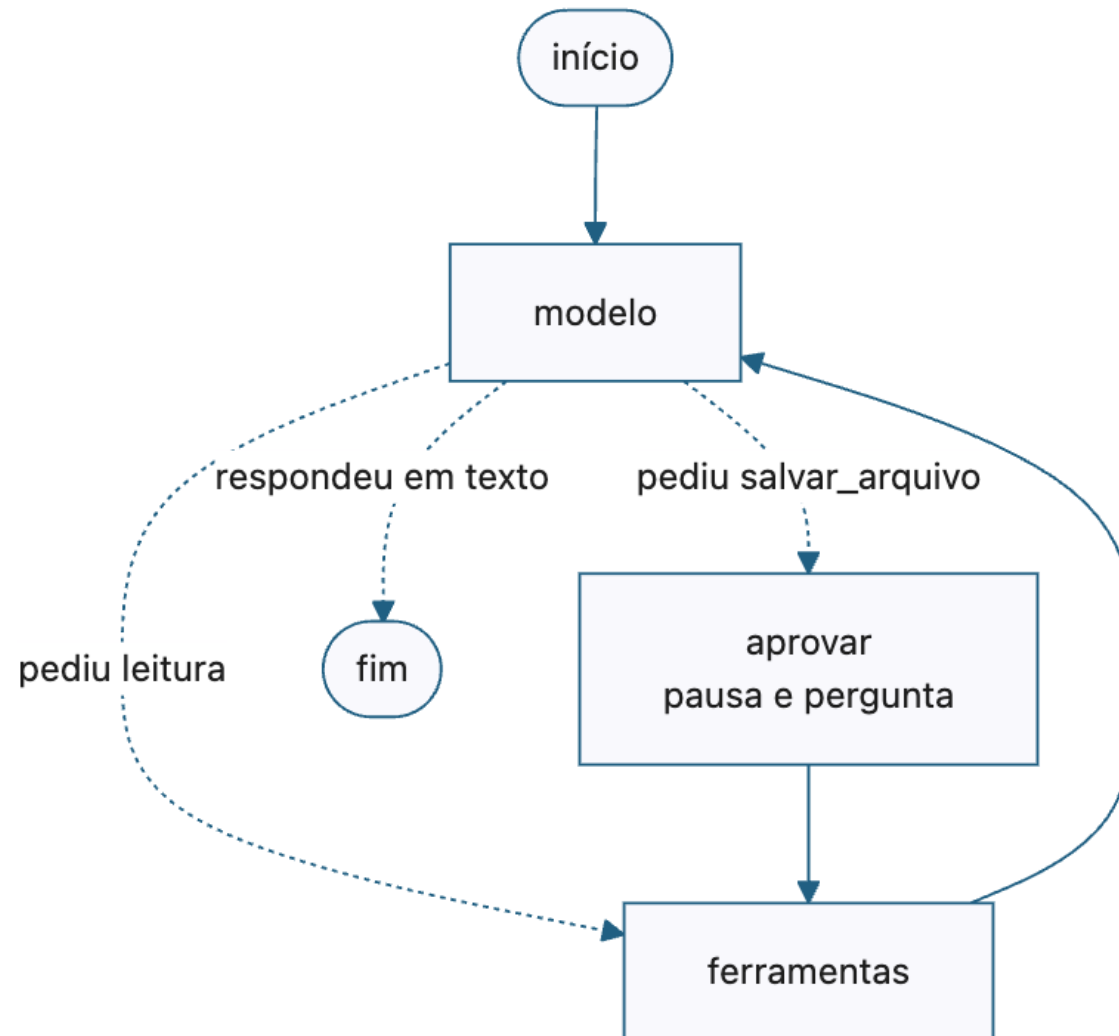
Exemplo 7 - Human in the loop



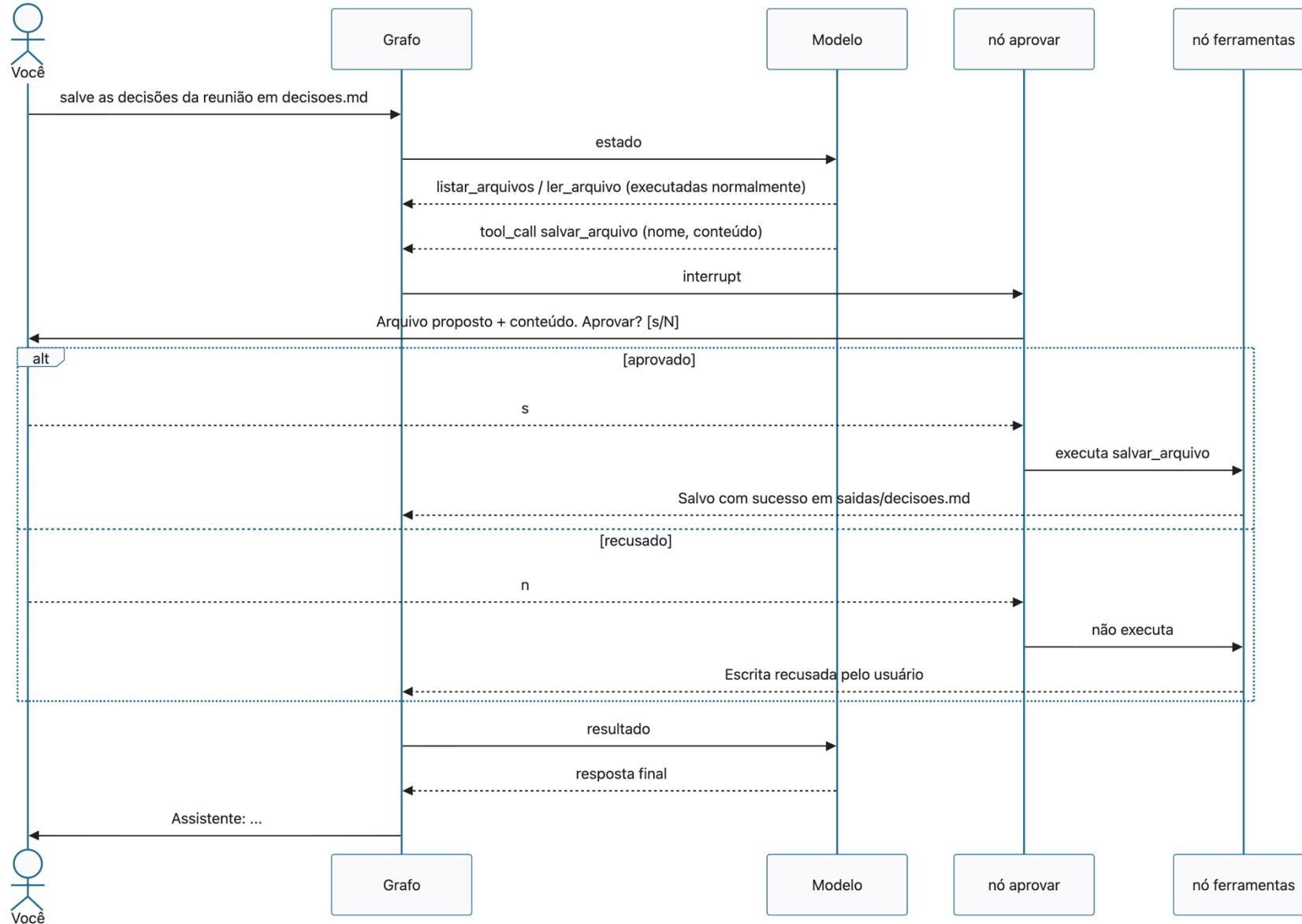
Em alguns fluxos, é necessária a interação/aprovação humana, como por exemplo no acesso a dados sensíveis, ou modificação de arquivos.

Esta interação humana é chamada de HIL (Human in the loop), onde o fluxo do agente é interrompido e pode ser modificado de acordo com a resposta do humano.

Exemplo 7 - Human in the loop



Exemplo 7 - Human in the loop





Hands on

Exemplo 8 - Hands on



No exemplo, o agente já enxerga a ferramenta "buscar_nos_arquivos", mas ela não está implementada.

Realize a implementação da ferramenta em "mcp_servers/exercicio.py" para que ela percorra os arquivos e devolva arquivo, linha e texto de cada ocorrência do termo de busca.

Teste perguntando algo como "Em quais arquivos aparece a palavra sensor?"



Referências



Referências

- <https://www.langchain.com/blog/how-to-build-a-custom-agent-harness>
- Let's build an AI Agent - Phil Nash - NDC Copenhagen 2026 https://www.youtube.com/watch?v=2r47_vCEkyo
- <https://www.anthropic.com/news/model-context-protocol>



UNISINOS